

Baby Redis

A from-scratch, layer-by-layer reimplementation of Redis's core mechanics in C++

Author: Aman Kajla | Language: C++17 | Environment: WSL (Ubuntu)

Baby Redis is an in-memory key-value store built in C++, implemented as a sequence of five architectural layers, each adding one capability found in production systems like Redis: an in-memory hash-map core, crash-durable persistence via an append-only log, memory-bounded LRU eviction, a TCP network interface, and finally key expiry (TTL), background expiry sweeping, request benchmarking, and thread-safe concurrent client handling.

Scope note: this project is not intended as a production alternative to Redis. Real Redis supports many additional data types, a highly-optimized single-threaded event loop, clustering, replication, and hardened persistence formats. Baby Redis instead implements the core mechanics that underlie such systems, from first principles, with each design decision documented below alongside the reasoning and trade-offs behind it.

Contents

Contents	2
Features	4
Getting Started	5
Prerequisites	5
Build	5
Run	5
Connect	5
Benchmark	5
Usage	6
Command Reference	6
Example Session	6
System Architecture	7
Layer 1 — In-Memory Store	8
Concepts & Tools Used	8
Design Rationale & Edge Cases	8
Layer 2 — Persistence (Append-Only Log)	8
Concepts & Tools Used	8
Verified Failure Case	9
Layer 3 — LRU Eviction	10
Data Structure Design	10
Correctness Details	10
Log-Eviction Consistency	10
Layer 4 — TCP Networking	11
Syscalls Used	11
The Byte-Stream Framing Problem	11
Sequential Multi-Client Support	11
Layer 5 — TTL, Active Expiry, Benchmarking, Concurrency	12

TTL & Lazy Expiry12

Active (Background) Expiry12

Concurrency: Thread-per-Connection + Mutex12

Benchmarking13

Known Limitations **14**

Design Philosophy **14**

Features

- In-memory key-value store with average $O(1)$ get, set, and delete operations
- Crash-durable persistence via an append-only log, replayed on startup through the live command path
- Memory-bounded LRU eviction (hash map + doubly linked list), all operations $O(1)$
- TCP network interface (raw POSIX sockets) supporting sequential and concurrent clients
- Key expiry (TTL) with both lazy (on-access) and active (background sweep) expiry
- Thread-per-connection concurrency with mutex-protected shared state
- A standalone benchmarking client measuring throughput and per-request latency

Getting Started

Prerequisites

A C++17-capable compiler (g++) and a POSIX-compliant environment. This project uses raw POSIX socket headers (<sys/socket.h>, <netinet/in.h>, <unistd.h>), which are unavailable under native Windows/MinGW. On Windows, use WSL (Windows Subsystem for Linux) running Ubuntu.

Build

```
g++ -std=c++17 -o babyredis main.cpp
```

Run

```
./babyredis
```

The server listens on port 8080. On startup, it replays babyredis.log (if present) to restore state from any previous run.

Connect

```
nc localhost 8080
```

Any TCP client works; netcat (nc) is used throughout development and testing. Multiple clients may connect at the same time.

Benchmark

```
g++ -std=c++17 -o benchmark benchmark.cpp  
./benchmark
```

Run with the server already running in a separate terminal. Reports throughput (ops/sec) and per-request latency (avg/min/max) over 1,000 SET commands.

Usage

Command Reference

Command	Syntax	Description
SET	SET key value [EX seconds]	Stores key. Values may contain spaces. Optional EX sets an expiry in seconds from now. A bare SET clears any existing TTL on the key.
GET	GET key	Returns the value for key, or NIL if the key is missing or has expired.
DEL	DEL key	Deletes key if present. Returns 1 if a key was deleted, 0 if it did not exist.
QUIT	QUIT	Ends the current client's session. The server continues running and accepting other connections.

Example Session

```
nc localhost 8080
SET greeting hello world
OK
GET greeting
hello world
SET session abc123 EX 30
OK
GET session
abc123
DEL greeting
1
GET greeting
NIL
QUIT
Quiting the Session!
```

After 30 seconds, GET session would return NIL as the key's TTL has elapsed.

System Architecture

Each layer wraps the layer beneath it without modifying it. The Layer 1 hash map remains the system's core data structure all the way through Layer 5; each successive layer adds a capability around it.

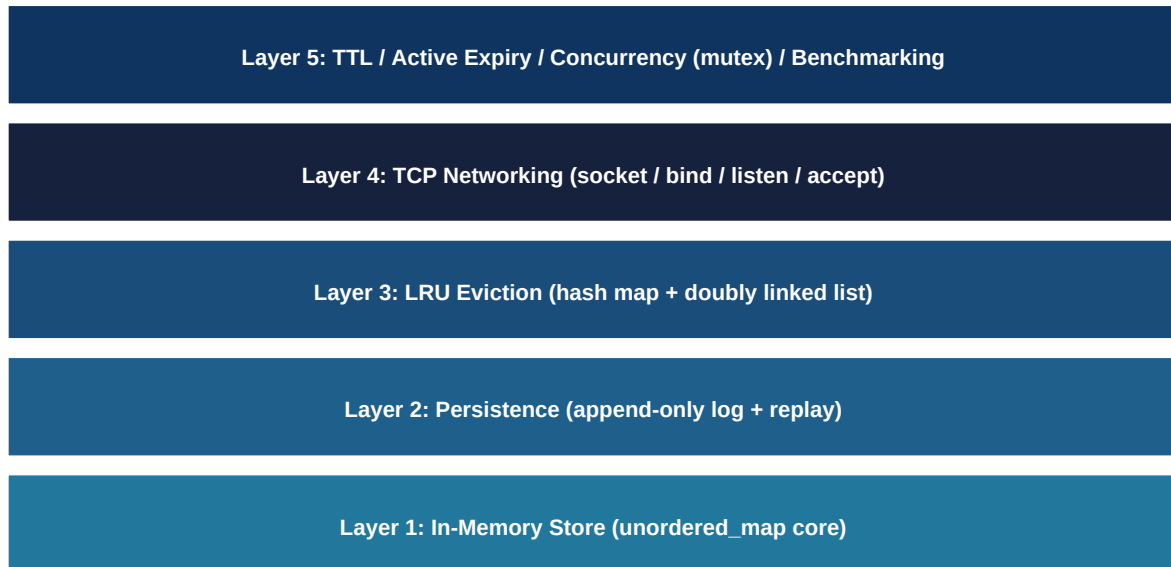


Figure 1: Layered architecture. Lower layers have no awareness of the layers built on top of them (e.g. the Layer 1 Store has no knowledge of persistence, networking, or concurrency), keeping each layer independently testable.

Layer 1 — In-Memory Store

A hash map wrapped in a read → parse → dispatch → execute command loop. The `Store` class knows nothing about I/O — it is a pure, testable data structure.

Concepts & Tools Used

- **`unordered_map<string,string>`** — average $O(1)$ get/set/erase by key. Chosen over `std::map` (a balanced tree, $O(\log n)$, keeps keys sorted — an ordering not needed for a plain key-value store).
- **`optional<string>`** — represents "this key may or may not exist" without abusing an empty string as a fake sentinel (an empty string is a valid value a user might store). `nullopt` = missing, any real string (including "") = found.
- **`.find()` instead of `operator[]`** — `data[key]` on an `unordered_map` *inserts* a default entry if the key is missing, a silent side effect for a read-only lookup. `.find()` returns an iterator without mutating the map.
- **`del(key)` returns a count, not "OK"** — matches Redis semantics: returns the number of keys actually deleted (0 or 1), making DEL naturally idempotent.
- **`getline(cin, ...)` vs `cin >>`** — `cin >>` stops at whitespace; `getline` reads a full line, which the command loop then parses itself via `istringstream`.

Design Rationale & Edge Cases

- **`unordered_map` vs. `map`**: `unordered_map` offers average $O(1)$ operations against `std::map`'s guaranteed-but-slower $O(\log n)$, at the cost of no ordering guarantee — an acceptable trade-off since key ordering is not required here.
- **DEL returns a count, not a status string** — this makes the operation naturally idempotent and machine-checkable (0 or 1 keys removed), rather than relying on string comparison for success/failure.
- **I/O-agnostic design**: because `Store` contains no I/O logic, its correctness can be verified independently of the command loop, network layer, or log — a property preserved through every subsequent layer.

Layer 2 — Persistence (Append-Only Log)

Every mutation (SET/DEL, never GET) is appended to a file. On startup, the log is replayed through the exact same command path used for live commands, rebuilding state.

Concepts & Tools Used

- **`ofstream logFile("babyredis.log", ios::app)`** — opened once, kept alive for the whole run, in append mode: new writes go to the end only, never overwriting existing content.
- **Append-only vs. in-place overwrite** — sequential appends are fast, and a crash mid-append can only corrupt the one line being written; everything before it stays intact. In-place rewriting risks corrupting previously-good bytes if a crash happens mid-rewrite.
- **flush vs. fsync** — `endl.flush()` pushes data out of the *program's* buffer to the OS (protects against program crashes). It does not guarantee the bytes are physically on disk — true power-loss durability needs `fsync`, a heavier syscall.
- **`writeChange(cache, logFile, line)`** — one function, two callers: the live command loop and startup replay both reuse it, avoiding two parsers silently drifting apart over time.

- **The nullptr-during-replay convention** — passing nullptr as the log pointer during replay suppresses re-logging (otherwise every restart would re-append replayed history into the log forever).

Verified Failure Case

A log line was deliberately truncated mid-write (`SET foo bar` → `SET foo b`) to simulate a crash mid-append. Replay silently applied it as a valid-looking but wrong command — `GET` returned `b`, not the original value and not `nil`. This confirms that a half-written line is not simply "missing" data; it can resemble a completely different, valid command and silently corrupt state. The mitigation used by production systems (not implemented here) is per-entry checksums or length-prefixes, as in Redis's own AOF format.

Layer 3 — LRU Eviction

Caps memory growth and keeps "hot" keys fast to access by evicting the least recently used key once capacity is exceeded. Every get/set is $O(1)$.

Data Structure Design

- **Hash map + doubly linked list** — need $O(1)$ key→node lookup (hash map) *and* $O(1)$ reordering by recency (a hash map has no concept of order). A single ordered structure (`std::map<key, timestamp>`) would cost $O(\log n)$ per touch and still need a separate structure for $O(1)$ lookup.
- **Doubly, not singly, linked** — removing an arbitrary node requires patching its *predecessor's* next pointer. A singly linked list has no prev pointer, so removal would need an $O(n)$ walk from the head.
- **Circular single-dummy sentinel** — one sentinel node whose next points to the MRU end and prev points to the LRU end. Every real node's prev/next always points to another real node or the dummy — never nullptr — so get/set/del/eviction never need special-case null checks.
- **Node stores its own key** — eviction hands you a `Node*` at the LRU end, but erasing the matching hash-map entry requires the key itself; a value cannot be reversed back into a map key.

Correctness Details

- `Capacity > map.size()`, not `>=` — `>=` would let one extra item slip in over capacity before eviction ever triggers.
- `assert(capacity > 0)` — guards a real corruption case: with capacity 0, eviction would call `unLink` on the dummy itself.
- Ordering constraint in `insertFront`: `dummy->next` (the current first real node) must be captured before overwriting `dummy->next`, or the old first node's identity is lost before its prev pointer is patched.

Log-Eviction Consistency

GET changes recency order but GET is never logged (Layer 2's rule). A GET that saves a key from live eviction was therefore invisible to replay, allowing replayed eviction order to diverge from live order after a crash.

Resolution: `set()` returns `optional<string>` — the evicted key, if any — so the caller (`writeChange`) can explicitly log a matching DEL. This keeps `LRUCache` itself I/O-agnostic, consistent with `Store`'s separation of concerns in Layer 1.

Layer 4 — TCP Networking

Swaps the input source from cin (a human typing) to a TCP socket (a network client). The command loop itself does not change — only where the bytes come from changes.

Syscalls Used

Call	Purpose
<code>socket(AF_INET, SOCK_STREAM, 0)</code>	Creates a communication endpoint, returns a file descriptor. Nothing bound/listening yet.
<code>setsockopt(..., SO_REUSEADDR ...)</code>	Lets the server rebind a port still in TIME_WAIT from a previous run, instead of failing 'Address already in use.'
<code>bind(sockfd, &addr, ...)</code>	Tells the OS to route traffic for this address/port to this socket.
<code>listen(sockfd, backlog)</code>	Flips the socket into passive mode: start queuing incoming connections.
<code>accept(sockfd, ...)</code>	Blocks until a client connects; returns a NEW fd dedicated to that one client.
<code>read(clientfd, buf, n) / write(...)</code>	Raw POSIX I/O — getline/cout do not work on a socket fd, since it is not a C++ stream object.
<code>close(clientfd)</code>	Releases the connection once a client's session ends (disconnect or QUIT).

The Byte-Stream Framing Problem

TCP has no concept of "messages," only an ordered stream of bytes. A single `read()` call might return a full command, a partial command, or multiple commands glued together. Solved with a persistent `string inputBuffer` (declared once per client, outside the read loop) that accumulates bytes across reads; an inner loop extracts and processes every complete line (`find( '\n' )`) currently sitting in the buffer before returning to `read()` for more.

Sequential Multi-Client Support

The server is structured as two nested loops: an outer loop calls `accept()` once per client, an inner loop serves that one client until `disconnect/QUIT`, then falls back to `accept()` for the next. `QUIT` breaks only the current client's session (via a boolean flag propagated up through both loops) instead of ending the whole process. `close(clientfd)` runs unconditionally after the inner loop, covering both exit paths with a single call.

Layer 5 — TTL, Active Expiry, Benchmarking, Concurrency

TTL & Lazy Expiry

- **Absolute, not relative, timestamps** — expiry is computed once as `now + duration` at SET time and stored as `optional<time_t>`. A relative duration would mean something different on replay (which could happen anytime after restart) than it did live — absolute timestamps mean the same instant regardless of when they're read.
- **optional<time_t> over a sentinel value** — mirrors Layer 1's `optional<string>` choice: self-documenting "no expiry" state rather than an implicit convention like a max-value sentinel.
- **Syntax**: `SET key value EX <seconds>`. Values may contain spaces (`SET greeting hello world` stores the full string); the parser tokenizes the remainder of the line and detects a trailing `EX <seconds>` pair independently of value length. A bare SET clears any existing TTL (matches real Redis's default; `KEEPTTL` would be the explicit opt-in to preserve it).
- **Lazy expiry** — checked only inside `get()`: if a node's expiry has a value and `now ≥ that value`, the key is deleted on the spot and treated as a miss.
- **Input validation** — duration parsing is validated (non-empty, all-digits) before calling `stoi()`, preventing malformed input such as `SET foo bar EX` from crashing the server.

Active (Background) Expiry

Pure lazy expiry has a gap: an expired key sitting mid-list (never GET'd) is invisible to eviction logic, which blindly evicts whatever sits at the LRU tail — even a still-valid key — while a stale expired key keeps occupying a slot. `sweepExpired()` resolves this by scanning every node once (expiry time is unrelated to recency order, so no early-exit shortcut is possible) and deleting any that have expired. It is invoked once per new client connection, immediately before `accept()`. A key implementation detail: the current node's `next` pointer must be saved before calling `del()` on it, since `del()` frees that node's memory — reading `node->next` afterward would be a use-after-free.

Concurrency: Thread-per-Connection + Mutex

- **Problem addressed** — prior to this, the server was single-threaded and blocking: while one client was connected, the server could not `accept()` a second, already-waiting client until the first disconnected. Correct sequential support existed, but not true simultaneous support.
- **Thread-per-connection** — each accepted client is handed off via `thread` `clientThread(handleClient, ...); clientThread.detach(); .detach()` (rather than `.join()`) lets the thread run independently, so the main loop returns to `accept()` immediately rather than blocking until that client finishes.
- **std::ref() vs. &** — `handleClient` takes `LRUCache&` by reference. `std::thread` copies its arguments by default; wrapping a reference-parameter argument in `std::ref(...)` makes the thread pass a genuine reference through rather than silently operating on a disconnected copy.
- **Mutex, narrowly scoped** — a single `std::mutex` protects only the one call that touches shared state (cache and log file), via a scoped `std::lock_guard` rather than manual `lock()/unlock()` calls. `lock_guard`'s destructor releases the lock automatically even if an exception is thrown mid-call, avoiding a permanently-leaked lock that manual `unlock()` calls are vulnerable to.
- **Failure mode without the mutex** — two threads calling `set()` simultaneously could interleave the multiple sequential pointer writes inside `unLink()/insertFront()` (patching `next`, then `prev`, then the dummy), corrupting

the linked list's structure — a race condition on a non-atomic, multi-step operation. In ACID terms, the mutex enforces Isolation.

Benchmarking

A separate client program (`benchmark.cpp`) connects via the same `socket/connect()` path a real client would use, then issues 1,000 SET commands back-to-back, timing each round-trip with `<chrono>`.

Metric	Before Concurrency	After Concurrency
Throughput	17,672.7 ops/sec	14,303.8 ops/sec
Avg latency	0.0554 ms	0.0682 ms
Min latency	0.0424 ms	0.0337 ms
Max latency	0.8048 ms	0.6447 ms

Interpretation: throughput measured slightly lower after threading, under a single-client, sequential-request benchmark. This is expected rather than a regression: a single client sending commands one at a time cannot exercise concurrency at all, so the only visible effect is the small fixed overhead of thread creation and mutex locking per request, with no throughput upside to offset it. Concurrency's benefit is realized under genuinely concurrent load — many simultaneous clients — which a multi-client benchmark (a natural next extension) would be needed to demonstrate.

Known Limitations

Documented openly, as clearly understood gaps in a working system provide more useful engineering context than an implied claim of completeness.

- No checksum or corruption-detection on log entries (see Layer 2's verified crash-mid-write failure case above; real Redis AOF uses length-prefixing for this).
- No log compaction implemented (design understood: periodically dump current keys as fresh SETs into a new file, then atomically swap it in — not built).
- LRU capacity is hardcoded (LRUCache cache(3) in main) — not configurable via CLI, config file, or environment variable.
- Listening port (8080) and connection backlog (1) are hardcoded.
- Benchmarking currently exercises a single client sequentially; a multi-client concurrent benchmark would better demonstrate threading's throughput characteristics under real concurrent load.

Design Philosophy

Baby Redis is not a production-competitive alternative to Redis — it deliberately omits the additional data types, the highly-optimized single-threaded event loop, clustering, replication, and hardened persistence formats that make production Redis suitable for large-scale deployment. Its value lies in the reverse direction: each mechanism — the hash map, the eviction list, the network syscalls, the durability trade-offs, the thread-safety model — was built and verified independently, producing a working, first-principles account of the engineering decisions such systems are built on. The project's thread-per-connection concurrency model, in particular, offers a direct and measured point of comparison against real Redis's single-threaded event-loop design.